

QUEEN'S UNIVERSITY BELFAST

MSc ARTIFICIAL INTELLIGENCE

Pre-course preparation plan

13 July - 15 September 2026

A nine-week programme for Python fluency, mathematical refresh, classical machine learning and knowledge engineering.

Duration	Target workload	Core output	Course start
9 weeks + buffer	12-14 hours/week	Code portfolio + report	15 September 2026

GOAL The central objective

Arrive able to write, inspect and debug Python independently; use the scientific Python stack; recover the mathematics behind core AI methods; and complete a small, reproducible machine-learning investigation without outsourcing the reasoning to AI.

How this document is intended to work

- [] Print it or use the editable Word version as a weekly checklist.
- [] Follow the specified sections of each resource rather than trying to complete every course or textbook.
- [] Complete the minimum outcome first. Stretch work is optional and should never block the next week.
- [] Keep all code in one Git repository so progress is visible and recoverable.

Planning note

This guide is aligned to the current QUB course page and the latest accessible official programme specification. Module content and assessment arrangements can change; use the module handbooks issued at enrolment as the final authority.

Contents

1	What to prioritise before September
2	Study method, weekly rhythm and AI guardrails
3	Baseline diagnostic
4	Nine-week study plan
5	Capstone project specification
6	Progress tracker
7	Final readiness assessment
8	Resource directory

Quick-start instructions

1. Today: create the repository, install a Python environment, and complete the diagnostic without AI.
2. Each Sunday: update the progress tracker, identify one recurring mistake, and choose the first task for Monday.
3. When time is tight: complete the minimum outcome and skip stretch work. Do not merge unfinished weeks together.
4. When stuck: use documentation, then request an explanation or hint from AI. Rebuild the solution from a blank file afterwards.

1. What to prioritise before September

The current programme structure places Foundations of AI, Machine Learning and Knowledge Engineering in the autumn term. The plan therefore prioritises the skills needed to survive those modules immediately. Computer Vision and Natural Language Processing are important, but deep learning receives only a short preview before the course begins.

Autumn area	Preparation needed	Your likely position	Priority
Python and scientific computing	Independent coding; testing; NumPy; pandas; plotting	Programming concepts transfer from JavaScript, but Python fluency is limited	Very high
Foundations of AI	Linear algebra; calculus; probability; optimisation; CSPs; MDPs	Strong engineering background, but likely rusty	High
Machine Learning	Model workflow; validation; metrics; classical algorithms; interpretation	Some data analysis experience, but limited formal ML	Very high
Knowledge Engineering	Logic; ontologies; knowledge graphs; Bayesian networks	Likely mostly new	High
Technical reporting	Reproducible experiments; critical discussion; limitations	A significant existing strength	Maintain
Deep learning	Tensors; training loop; automatic differentiation	Mostly new, but taught later	Medium

Target capability by 15 September

- [] Write and organise a small Python project without AI deciding its structure.
- [] Read, clean, transform and visualise tabular data using NumPy and pandas.
- [] Explain and apply the core mathematics behind linear models, PCA and optimisation.
- [] Build leak-free scikit-learn pipelines and evaluate models with defensible metrics.
- [] Understand the vocabulary of logic, ontologies, CSPs and MDPs well enough to follow postgraduate lectures.
- [] Produce a concise technical report that distinguishes results, interpretation, limitations and ethical risk.

IMPORTANT Do not optimise for course completion statistics

The goal is not to collect certificates or finish every video. A smaller number of concepts that you can reproduce, explain and apply independently will be far more useful than broad passive exposure.

2. Study method, weekly rhythm and AI guardrails

Recommended weekly rhythm

Use six flexible blocks rather than relying on a single long weekend session.

Block	Duration	Purpose
A	2 hours	New concept and worked examples
B	2 hours	Guided exercises
C	2 hours	Independent implementation
D	2 hours	Mathematics or theory problems
E	3-4 hours	Weekly project and tests
F	1-2 hours	Review, error log and no-AI retrieval

Structure of a two-hour session

- 15 minutes - retrieve the previous session from memory; do not open notes immediately.
- 35 minutes - learn one tightly defined concept.
- 60 minutes - solve problems or write code.
- 10 minutes - record mistakes, write the next task, and commit useful work.

AI-assisted coding protocol

1. Attempt the task for 25-30 minutes before asking AI for help.
2. Check official documentation before requesting generated code.
3. Ask for a hint, explanation, test case or code review rather than a full implementation.
4. After receiving help, close the answer and recreate the solution from a blank file.
5. Complete one 90-minute no-AI exercise each week. Documentation is allowed.
6. Record every material AI intervention in `ai_log.md`: the problem, the help requested, the misunderstanding and whether you can now reproduce the solution.

```
msc-ai-prep/
├── 01_python/
├── 02_numpy_pandas/
├── 03_linear_algebra/
├── 04_calculus_optimisation/
├── 05_probability_statistics/
├── 06_machine_learning/
├── 07_knowledge_engineering/
├── 08_advanced_ml/
├── 09_ai_foundations/
├── capstone/
├── notes/
└── ai_log.md
```

RECOVERY RULE The rule for falling behind

Do not extend a week indefinitely. Complete its minimum outcome, note the unfinished stretch items, and move on. The sequence matters more than polishing every exercise.

3. Baseline diagnostic

Complete this before using the week-by-week resources. Do not use AI.

A. Python diagnostic - 90 minutes

Write a command-line program that:

- ☐ reads a CSV containing experimental results;
- ☐ checks that required columns are present;
- ☐ reports or skips invalid rows;
- ☐ groups results by experimental condition;
- ☐ calculates count, mean, minimum and maximum for each group;
- ☐ writes the summary to a new CSV; and
- ☐ includes at least three automated tests.

First attempt: standard library only. Afterwards, note which parts were difficult because of Python syntax and which reflected a broader programming gap.

B. Mathematics diagnostic - 60 minutes

- ☐ Multiply two small matrices.
- ☐ Solve a two-variable linear system.
- ☐ Differentiate a polynomial and an exponential function.
- ☐ Find both partial derivatives of $f(x, y) = x^2 + 3xy + y^2$.
- ☐ Calculate a conditional probability using Bayes' theorem.
- ☐ Explain variance and covariance in your own words.

C. Machine-learning concepts - 30 minutes

- ☐ Define training, validation and test data.
- ☐ Define overfitting.
- ☐ Define regularisation.
- ☐ Define data leakage.
- ☐ Define classification versus regression.
- ☐ Define precision versus recall.
- ☐ Define cross-validation.

Score the result

Rating	Meaning	Action
Green	Solved and explained independently.	Follow the normal plan.
Amber	Understood after thought or checking notes.	Add one extra practice block.
Red	Did not know how to begin.	Shift 2-3 hours toward this area during the relevant week.

OUTPUT Diagnostic outcome to record

Write one sentence for each area: "I can already...", "I need to refresh...", and "I cannot yet...". Keep this page and repeat the diagnostic on 12 September.

Week 1: Python fundamentals and independent coding

Dates	Target time	Primary focus	End-of-week evidence
13-19 July	12-14 hours	Python fluency	Tested CSV experiment-summary CLI

Python is the working language for almost every later topic. This week is about replacing recognition of generated code with the ability to build and debug small programs yourself.

Learning targets

- ☐ Core syntax: variables, strings, lists, tuples, dictionaries and sets.
- ☐ Control flow: conditionals, loops and comprehensions.
- ☐ Functions, parameters, return values, scope and type hints.
- ☐ Exceptions, file I/O, imports, modules and virtual environments.
- ☐ Basic classes and dataclasses.
- ☐ Testing with pytest and reading tracebacks.

Resources - use these exact sections

Role	Resource	Use this part
Primary	CS50P - Introduction to Programming with Python	Use Weeks 0-6 selectively: Functions; Conditionals; Loops; Exceptions; Libraries; Unit Tests; File I/O.
Reference	The Python Tutorial	Use sections 3-9 when syntax or language behaviour is unclear.
Testing	pytest - Get Started	Follow the first test, exception testing and test discovery examples.

Build and practise

1. Create `experiment_summary` as a command-line program.
2. Separate parsing, calculations and output into modules.
3. Validate input and produce useful error messages.
4. Add at least six tests, including malformed data and an empty file.
5. Write a README with setup and usage instructions.

Minimum outcome A working program split into functions, with at least three passing tests and no AI-generated implementation retained without reconstruction.	Stretch work Add type hints throughout, a dataclass for a result row, and command-line options using <code>argparse</code> .
---	--

End-of-week self-check

- ☐ Create and activate a virtual environment.
- ☐ Explain mutable versus immutable objects.
- ☐ Choose appropriately between list, tuple, dictionary and set.
- ☐ Read a traceback and identify the failing line.
- ☐ Run pytest and interpret a failed assertion.

Week 2: NumPy, pandas and data analysis

Dates	Target time	Primary focus	End-of-week evidence
20-26 July	12-14 hours	Scientific Python	Reproducible exploratory-data-analysis notebook

Scientific Python changes how data problems are expressed. You need to become comfortable with arrays, tabular transformations and plotting before attempting machine learning.

Learning targets

- ☐ NumPy arrays, dimensions, shapes, dtypes and indexing.
- ☐ Boolean masks, reshaping, transposition and broadcasting.
- ☐ Vectorised operations and random-number generation.
- ☐ pandas Series and DataFrames; selection with loc and iloc.
- ☐ Missing values, grouping, aggregation, merging and derived columns.
- ☐ Clear plots and separation of exploration from reusable code.

Resources - use these exact sections

Role	Resource	Use this part
Primary	NumPy: Absolute Basics for Beginners	Complete array creation, indexing, shape, axes, basic operations and broadcasting.
Primary	pandas Getting Started Tutorials	Work through tutorials 1-6: tabular data, reading/writing, selection, plots, derived columns and statistics.
Reference	Matplotlib Quick Start Guide	Use for figures, axes, labels and saving plots.

Build and practise

1. Choose a small scientific or health-related CSV dataset.
2. Create a data-quality summary: shape, types, duplicates and missing values.
3. Produce descriptive statistics and at least four meaningful plots.
4. Write a short interpretation beneath each plot.
5. Move repeated cleaning logic into a Python module and call it from the notebook.

Minimum outcome A notebook that runs from top to bottom, creates four labelled plots and states at least three limitations or potential confounders.	Stretch work Write unit tests for the cleaning functions and export a cleaned dataset plus a machine-readable summary JSON.
--	---

End-of-week self-check

- ☐ Explain array shape and axis without looking them up.
- ☐ Use a Boolean mask and a groupby aggregation.
- ☐ Explain broadcasting with a small example.
- ☐ Distinguish loc from iloc.
- ☐ Explain why notebooks should not contain all reusable logic.

Week 3: Linear algebra for AI

Dates	Target time	Primary focus	End-of-week evidence
27 July-2 August	12-14 hours	Vectors and matrices	PCA from first principles notebook

Linear algebra is the language of data representation, transformations and many machine-learning algorithms. Prioritise geometric understanding as well as calculation.

Learning targets

- [] Scalars, vectors, matrices and tensors.
- [] Dot products, norms, distance and matrix multiplication.
- [] Systems of linear equations; rank, span, basis and independence.
- [] Orthogonality, projections and least squares intuition.
- [] Eigenvalues, eigenvectors and covariance matrices.
- [] Singular value decomposition and principal component analysis.

Resources - use these exact sections

Role	Resource	Use this part
Intuition	3Blue1Brown - Essence of Linear Algebra	Watch chapters 1-9 and the eigenvalue chapter. Pause and reproduce examples on paper.
Primary text	Mathematics for Machine Learning	Read Chapters 2, 3 and 4 selectively: linear algebra, analytic geometry and matrix decompositions.
Practice	MIT OCW - Linear Algebra	Use problem sets for systems, subspaces, orthogonality and eigenvalues when you need more practice.

Build and practise

1. Generate a correlated two-dimensional dataset.
2. Standardise the features.
3. Calculate its covariance matrix.
4. Find eigenvalues and eigenvectors with NumPy.
5. Project the data onto principal components.
6. Compare the result with scikit-learn PCA and explain any sign differences.

Minimum outcome

A correct two-dimensional PCA implementation with a written explanation of covariance, principal directions and information loss.

Stretch work

Generalise the implementation to n dimensions and plot explained variance for a higher-dimensional dataset.

End-of-week self-check

- [] Interpret matrix multiplication as a transformation.
- [] Explain span, basis and rank.
- [] Explain why eigenvectors are special directions.
- [] Connect covariance to PCA.
- [] State why feature scaling can change PCA results.

Week 4: Calculus and optimisation

Dates	Target time	Primary focus	End-of-week evidence
3-9 August	12-14 hours	Gradients and learning	Linear regression trained with gradient descent

You do not need proof-heavy calculus before September, but you must understand derivatives as rates of change and gradients as directions for changing a loss function.

Learning targets

- ☐ Derivatives, product rule and chain rule.
- ☐ Partial derivatives and gradients.
- ☐ Directional derivatives; Jacobian and Hessian intuition.
- ☐ Local and global minima; convexity.
- ☐ Loss functions and parameter optimisation.
- ☐ Batch, stochastic and mini-batch gradient descent.
- ☐ Analytical versus numerical gradients.

Resources - use these exact sections

Role	Resource	Use this part
Intuition	3Blue1Brown - Essence of Calculus	Watch derivatives, chain rule and partial derivatives/gradient episodes.
Primary text	Mathematics for Machine Learning	Read Chapter 5 (Vector Calculus) and Chapter 7 (Continuous Optimisation) selectively.
Practice	MIT OCW - Multivariable Calculus	Use Unit 2 materials on partial derivatives, gradients and optimisation.

Build and practise

1. Generate or load a simple regression dataset.
2. Define mean squared error.
3. Derive gradients for the weight and bias on paper.
4. Implement gradient descent using NumPy.
5. Plot loss against iteration and compare coefficients with scikit-learn.
6. Add a finite-difference gradient check.

Minimum outcome A working gradient-descent implementation plus a one-page derivation and an explanation of the learning-rate effect.	Stretch work Implement mini-batch gradient descent and compare convergence across three learning rates.
--	---

End-of-week self-check

- ☐ Differentiate a simple composite function.
- ☐ Calculate a two-variable gradient.
- ☐ Explain what a learning rate controls.
- ☐ Diagnose divergence or oscillation from a loss curve.
- ☐ Explain why scaling can improve optimisation.

Week 5: Probability and statistics

Dates	Target time	Primary focus	End-of-week evidence
10-16 August	12-14 hours	Uncertainty and inference	Simulation notebook and Gaussian Naive Bayes

Probability supports classification, Bayesian reasoning, uncertainty and model evaluation. Your diagnostics background gives useful intuition, but the formal notation needs to become comfortable again.

Learning targets

- ☐ Events, random variables and common distributions.
- ☐ Expectation, variance, covariance and correlation.
- ☐ Joint, marginal and conditional probability.
- ☐ Independence and conditional independence.
- ☐ Bayes' theorem and diagnostic-test base rates.
- ☐ Normal and multivariate normal distributions.
- ☐ Likelihood, maximum likelihood, priors and posteriors.
- ☐ Law of large numbers and confidence intervals.

Resources - use these exact sections

Role	Resource	Use this part
Primary	MIT OCW - Introduction to Probability and Statistics	Use units on conditioning, random variables, expectation, distributions, Bayesian inference and confidence intervals.
Reference	Mathematics for Machine Learning	Read Chapter 6 (Probability and Distributions).
Interactive	Seeing Theory	Use Basic Probability, Compound Probability and Distributions for visual reinforcement.

Build and practise

1. Simulate repeated coin tosses to show the law of large numbers.
2. Visualise normal distributions with different means and variances.
3. Generate correlated multivariate Gaussian data and estimate its covariance.
4. Solve a diagnostic-testing Bayes example with realistic low prevalence.
5. Implement a basic Gaussian Naive Bayes classifier from scratch.

Minimum outcome A reproducible simulation notebook and a written explanation of why $P(A B)$ differs from $P(B A)$.	Stretch work Compare your Naive Bayes probabilities and predictions with scikit-learn and investigate disagreement.
--	---

End-of-week self-check

- ☐ Explain expectation versus sample mean.
- ☐ Explain covariance versus correlation.
- ☐ Calculate a posterior probability with Bayes' theorem.
- ☐ Explain prior, likelihood and posterior.
- ☐ Explain the base-rate problem in diagnostic testing.

Week 6: Classical machine learning and experiment design

Dates	Target time	Primary focus	End-of-week evidence
17-23 August	14 hours	Reliable ML workflow	First complete capstone experiment

This is the most important applied week. The objective is not to memorise every algorithm, but to build a defensible experimental workflow that avoids leakage and misleading evaluation.

Learning targets

- ☐ Features, targets, baselines and train/test splitting.
- ☐ Cross-validation and preprocessing pipelines.
- ☐ Data leakage and reproducibility.
- ☐ Linear and logistic regression; k-nearest neighbours.
- ☐ Decision trees, random forests and support-vector machines.
- ☐ Regularisation and the bias-variance trade-off.
- ☐ Confusion matrices; accuracy, precision, recall, specificity, F1 and ROC-AUC.
- ☐ Hyperparameter tuning and model interpretation.

Resources - use these exact sections

Role	Resource	Use this part
Primary	scikit-learn MOOC	Prioritise predictive modelling pipelines, preprocessing, cross-validation, linear models, trees, ensembles and metrics.
Reference	scikit-learn - Common Pitfalls	Read inconsistent preprocessing, data leakage, randomness and reproducibility.
Dataset	Breast Cancer Wisconsin dataset	Use the built-in dataset for the capstone unless you already have a manageable alternative.

Build and practise

1. Create a held-out test set before model selection.
2. Compare a dummy baseline, logistic regression, k-NN, SVM and random forest.
3. Put all preprocessing inside pipelines.
4. Use cross-validation on the training set.
5. Choose metrics based on the problem rather than convenience.
6. Produce a comparison table, confusion matrices and initial error analysis.

Minimum outcome A script or notebook that reruns cleanly and compares at least three real models against a dummy baseline without leakage.	Stretch work Add nested or repeated cross-validation, calibration analysis or permutation importance.
--	---

End-of-week self-check

- ☐ Explain why a baseline is necessary.
- ☐ Explain why preprocessing belongs inside cross-validation.
- ☐ Identify overfitting from train and validation results.
- ☐ Justify precision, recall or specificity for a health-related problem.
- ☐ Keep the held-out test set untouched until the final comparison.

Week 7: Knowledge engineering, logic and ontologies

Dates	Target time	Primary focus	End-of-week evidence
24-30 August	12-14 hours	Symbolic AI	Diagnostic-laboratory ontology in Protégé

This is likely to be the least familiar autumn area. The target is vocabulary, formal reasoning and hands-on ontology modelling rather than complete mastery of logic.

Learning targets

- [] Propositions, truth tables, implication and equivalence.
- [] Validity, satisfiability and entailment.
- [] First-order logic: predicates, variables and quantifiers.
- [] Facts, rules, classes, instances and relations.
- [] Ontologies, taxonomies and knowledge graphs.
- [] Open-world versus closed-world assumptions.
- [] Bayesian networks and conditional independence.
- [] Decision theory: actions, outcomes, probability and utility.

Resources - use these exact sections

Role	Resource	Use this part
Primary	W3C OWL 2 Primer	Read the introduction, classes and instances, properties, class hierarchies and advanced class relationships.
Tool	Protégé Desktop	Install Protégé and follow its getting-started materials while building the ontology.
AI reference	AIMA - Artificial Intelligence: A Modern Approach resources	Use the logic, knowledge representation and probabilistic reasoning topic links as reference.

Build and practise

1. Model DiagnosticTest, SampleType, Pathogen, AssayMethod, Result and Instrument.
2. Include LAMP and PCR as method classes or individuals with a defensible design.
3. Add relationships such as detectsPathogen, usesSample and requiresInstrument.
4. Add domains, ranges, disjoint classes and restrictions.
5. Create one inconsistency and use a reasoner to detect it.
6. Document at least three inferences produced by the reasoner.

Minimum outcome A saved ontology with a coherent hierarchy, properties, restrictions and one explained reasoning result.	Stretch work Query the ontology with SPARQL or export RDF and inspect it using a Python RDF library.
--	--

End-of-week self-check

- [] Construct a truth table.
- [] Translate a simple English sentence into predicate notation.
- [] Explain class versus instance.
- [] Explain what an ontology adds beyond a spreadsheet.
- [] Explain the open-world assumption.

Week 8: Unsupervised learning, Bayesian ML and deep-learning preview

Dates	Target time	Primary focus	End-of-week evidence
31 August-6 September	12-14 hours	Broader ML landscape	Clustering/PCA comparison and first PyTorch network

These topics broaden the map of machine learning. Aim for conceptual familiarity with advanced Bayesian methods and basic competence with a neural-network training loop.

Learning targets

- ☐ k-means, hierarchical clustering and density-based clustering.
- ☐ Cluster interpretation and limitations.
- ☐ PCA as dimension reduction.
- ☐ Frequentist versus Bayesian model fitting.
- ☐ Bayesian linear regression, Gaussian-process and MCMC intuition.
- ☐ Tensors, layers, activations and forward passes.
- ☐ Loss functions, backpropagation and automatic differentiation.

Resources - use these exact sections

Role	Resource	Use this part
Primary ML	scikit-learn - Clustering and Decomposition	Read the overview and practical guidance for clustering and decomposition/PCA.
Deep learning	PyTorch - Learn the Basics	Complete tensors, datasets, model building, autograd and optimisation.
Bayesian overview	Probabilistic Machine Learning: An Introduction	Skim introductory Bayesian modelling, Gaussian processes and MCMC sections; do not attempt full coverage.

Build and practise

1. Apply PCA and two clustering methods to a suitable dataset.
2. Compare results and explain why cluster labels are not ground truth.
3. Train a small multilayer perceptron using the PyTorch beginner workflow.
4. Annotate the training loop: inputs, parameters, outputs, loss, gradient calculation and parameter update.

Minimum outcome One clear unsupervised-learning comparison and one small PyTorch model that you can explain line by line.	Stretch work Visualise learned representations or compare the network with a classical baseline.
---	--

End-of-week self-check

- ☐ Explain when clustering might be misleading.
- ☐ Explain PCA variance reduction.
- ☐ Identify every stage of a PyTorch training loop.
- ☐ Explain the purpose of autograd.
- ☐ Give a high-level reason MCMC is used.

Week 9: Constraint satisfaction, MDPs and consolidation

Dates	Target time	Primary focus	End-of-week evidence
7-13 September	12-14 hours	AI problem formulation	CSP, grid-world MDP and final capstone report

Finish by connecting mathematical concepts to classical AI problem formulations, then consolidate the entire programme through a timed assessment and polished capstone.

Learning targets

- ☐ CSP variables, domains, constraints and search trees.
- ☐ Backtracking, constraint propagation and variable-ordering heuristics.
- ☐ MDP states, actions, transition probabilities, rewards and policies.
- ☐ Discount factors, state values and Bellman equations.
- ☐ Value iteration.
- ☐ Technical reporting, limitations, ethics and reproducibility.

Resources - use these exact sections

Role	Resource	Use this part
Primary	AIMA Python repository	Use csp.py and mdp.py examples after attempting your own small implementations.
Reference	AIMA book resources	Review constraint satisfaction, probabilistic reasoning and sequential decision topics.
Writing	The Turing Way - Reproducible Research	Use the reproducibility checklist when finalising the capstone repository and report.

Build and practise

1. Implement map colouring or a small timetable allocation CSP.
2. Implement value iteration for a small grid world.
3. Clean the capstone repository and fix reproducibility problems.
4. Write a 1,500-2,000 word coursework-style report.
5. Complete the final no-AI assessment.

Minimum outcome A functioning CSP, a tabular value-iteration example, and a complete capstone report with methods, results, limitations and ethics.	Stretch work Add policy visualisation, sensitivity to discount factor, or a short presentation deck summarising the capstone.
---	---

End-of-week self-check

- ☐ Identify variables, domains and constraints in a new problem.
- ☐ Explain state, action, reward, policy and discount factor.
- ☐ Describe value iteration at a high level.
- ☐ Run the capstone from a clean environment.
- ☐ State what you would change with more data or time.

14 September: buffer, setup and stop

Do not use the final day for an all-day cram. The aim is to remove practical friction and arrive rested.

- [] Repeat only the red items from the final diagnostic.
- [] Confirm that Python, your editor, Jupyter and Git work correctly.
- [] Create a clean environment and rerun the capstone from its README.
- [] Push the final preparation repository and create a backup.
- [] Organise notes by module: Foundations, Machine Learning and Knowledge Engineering.
- [] Write a one-page list of unresolved questions to take into the first teaching week.
- [] Stop at a reasonable time and sleep normally.

PERSPECTIVE What success looks like on 14 September

You will still have gaps. Readiness means that the basic tools and notation are familiar enough that lectures can add depth rather than introduce an entirely new language.

5. Capstone project specification

A small, rigorous project is more valuable than a large, unfinished one.

Recommended question

How do several classical classification models compare on the scikit-learn Breast Cancer Wisconsin dataset when evaluated using a leak-free, reproducible workflow?

This is an educational exercise, not a clinically valid diagnostic study. The dataset is small, historical and unsuitable for medical deployment claims.

Required models

- ☐ Dummy classifier baseline
- ☐ Logistic regression
- ☐ k-nearest neighbours
- ☐ Support-vector machine
- ☐ Random forest

Required workflow

1. Define the prediction task and target clearly.
2. Create a held-out test set before model comparison.
3. Use pipelines for scaling or other preprocessing.
4. Perform cross-validation only on the training data.
5. Choose and justify primary and secondary metrics.
6. Report uncertainty or variability across folds, not only a single score.
7. Compare final models on the untouched test set once.
8. Inspect errors and discuss plausible causes.
9. State dataset limitations, fairness concerns and deployment risks.
10. Make the repository runnable from a clean environment.

Report structure - 1,500 to 2,000 words

Section	Purpose	Suggested length
1. Problem and context	Define the task, intended use and what the project does not claim.	150-200
2. Data	Describe samples, features, target, data quality and limitations.	200-250
3. Methods	Explain split strategy, preprocessing, models, tuning and metrics.	350-450
4. Results	Present cross-validation and test results with clear tables/plots.	300-400
5. Error analysis	Examine mistakes and model behaviour.	200-250
6. Limitations and ethics	Discuss sample size, representativeness, leakage, fairness and clinical risk.	250-350
7. Conclusion	State what was learned and what should be done next.	100-150

Repository acceptance test

- ☐ A new user can install dependencies from documented instructions.
- ☐ Random seeds are controlled where relevant.
- ☐ The code creates results without manual notebook state.
- ☐ Plots have titles, labels and readable legends.
- ☐ No preprocessing is fitted on the full dataset before cross-validation.
- ☐ The README states the educational and non-clinical nature of the work.

6. Progress tracker

Update this once per week. Evidence should be a file path, commit, notebook or report section - not just "watched videos".

Week	Main deliverable	Evidence / file	Status
0	Baseline diagnostic	notes/diagnostic_2026-07-13.md	☐ Not started ☐ Active ☐ Done
1	Python experiment-summary CLI	01_python/experiment_summary/	☐ Not started ☐ Active ☐ Done
2	Exploratory data analysis	02_numpy_pandas/eda.ipynb	☐ Not started ☐ Active ☐ Done
3	PCA from first principles	03_linear_algebra/pca.ipynb	☐ Not started ☐ Active ☐ Done
4	Gradient-descent regression	04_calculus_optimisation/linear_regression.py	☐ Not started ☐ Active ☐ Done
5	Probability simulations + Naive Bayes	05_probability_statistics/	☐ Not started ☐ Active ☐ Done
6	Capstone model comparison	capstone/experiments/	☐ Not started ☐ Active ☐ Done
7	Protégé ontology	07_knowledge_engineering/diagnostics.owl	☐ Not started ☐ Active ☐ Done
8	Clustering/PCA + PyTorch network	08_advanced_ml/	☐ Not started ☐ Active ☐ Done
9	CSP, MDP and final report	09_ai_foundations/ + capstone/report.pdf	☐ Not started ☐ Active ☐ Done

Weekly reflection

Week	Most important thing learned	Recurring error or misconception	First task next week
1			
2			
3			
4			
5			
6			

Week	Most important thing learned	Recurring error or misconception	First task next week
7			
8			
9			

Hours tracker

Week	Target	Actual	No-AI practice completed?	Adjustment for next week
1	12-14		☐ Yes ☐ No	
2	12-14		☐ Yes ☐ No	
3	12-14		☐ Yes ☐ No	
4	12-14		☐ Yes ☐ No	
5	12-14		☐ Yes ☐ No	
6	12-14		☐ Yes ☐ No	
7	12-14		☐ Yes ☐ No	
8	12-14		☐ Yes ☐ No	
9	12-14		☐ Yes ☐ No	

7. Final readiness assessment

Complete on 12 or 13 September using documentation but no AI-generated code.

Three-hour assessment

Time	Task	Evidence
60 min	Python data-processing task from an unfamiliar CSV: validate, transform, summarise and test.	Working files and tests.
60 min	Paper mathematics: matrices, partial derivatives, gradient, Bayes theorem, covariance and one PCA explanation.	Scanned or typed answers.
60 min	Train and evaluate a basic ML pipeline from an unfamiliar CSV; justify the split, preprocessing and metrics.	Runnable notebook or script.

Readiness checklist

- ☐ Write a 150-300 line Python project without AI deciding its structure.
- ☐ Use virtual environments, modules, tests and Git.
- ☐ Load, clean, transform and visualise tabular data.
- ☐ Work comfortably with NumPy shapes and matrix operations.
- ☐ Explain eigenvectors, gradients, Bayes' theorem and covariance.
- ☐ Implement linear regression and gradient descent using NumPy.
- ☐ Build a leak-free scikit-learn pipeline.
- ☐ Compare models with cross-validation and suitable metrics.
- ☐ Explain overfitting, regularisation and bias-variance.
- ☐ Construct basic logical statements and a small OWL ontology.
- ☐ Explain the components of a CSP and an MDP.
- ☐ Train a simple PyTorch network and explain its training loop.
- ☐ Produce a reproducible technical report with limitations and ethical considerations.

TARGET How to interpret the result

You do not need every item to feel effortless. Green on Python workflow and core ML, with amber rather than red on the mathematics and symbolic-AI vocabulary, is a strong starting position.

8. Resource directory

All links are clickable in the Word and PDF versions.

Topic	Resource	Purpose
Course information	QUB MSc Artificial Intelligence course page	Official overview, course structure, modules and entry requirements.
Course information	QUB official programme specification	Current accessible formal programme aims, learning outcomes, module terms and assessment pattern.
Python	CS50P	Structured Python lessons and exercises.
Python	Official Python Tutorial	Language reference for programmers new to Python.
Testing	pytest Get Started	Testing basics and test discovery.
Scientific Python	NumPy Absolute Basics	Arrays, shapes, indexing, operations and broadcasting.
Scientific Python	pandas Intro Tutorials	DataFrames, files, selection, plots and aggregation.
Plotting	Matplotlib Quick Start	Figure and axes fundamentals.
Mathematics	Mathematics for Machine Learning	Linear algebra, calculus, probability and optimisation in one free text.
Linear algebra	3Blue1Brown Essence of Linear Algebra	Visual intuition for vectors, transformations and eigenvectors.
Linear algebra	MIT OCW Linear Algebra	Lectures, notes and problem sets.
Calculus	3Blue1Brown Essence of Calculus	Visual intuition for derivatives, chain rule and gradients.
Calculus	MIT OCW Multivariable Calculus	Partial derivatives, gradients and optimisation practice.
Probability	MIT OCW Probability and Statistics	Conditioning, distributions, inference and confidence intervals.
Probability	Seeing Theory	Interactive visual probability and statistics.
Machine learning	scikit-learn MOOC	Practical modelling workflow, validation, pipelines and algorithms.
Machine learning	scikit-learn Common Pitfalls	Leakage, preprocessing and reproducibility.
Unsupervised ML	scikit-learn Unsupervised Learning Guide	Clustering and dimensionality reduction.
Knowledge engineering	W3C OWL 2 Primer	Formal introduction to OWL ontologies.
Knowledge engineering	Protégé	Ontology editor and reasoning environment.
Classical AI	AIMA resources	Logic, CSP, probabilistic reasoning and decision-making references.

Topic	Resource	Purpose
Classical AI	AIMA Python	Reference implementations for CSPs, MDPs and other AI algorithms.
Deep learning	PyTorch Learn the Basics	Tensors, models, autograd and optimisation.
Bayesian ML	Probabilistic Machine Learning	Free advanced reference; skim only before the course.
Reproducibility	The Turing Way	Practical reproducible-research guidance.

Resource discipline

- [] Use the primary resource named in each week first.
- [] Use reference material only to resolve a specific confusion.
- [] Do not add a new course because the current explanation feels difficult; attempt exercises and seek a second explanation only after identifying the exact gap.
- [] Save useful links and notes in the repository README so they are available when the MSc begins.

Build fluency first. Then build depth.

The purpose of this plan is to make the opening weeks of the MSc challenging but intelligible - not to complete the MSc before it starts.